

CSS II — Selectores avanzados, box model, Flexbox y calidad de código

En esta sesión

En esta sesión vas a avanzar desde el CSS base hacia un CSS más útil para trabajar en interfaces reales.

Vas a estudiar:

- selectores más precisos;
- pseudo-clases y pseudo-elementos;
- box model, dimensiones, `overflow` y bordes;
- Flexbox como sistema de alineación y distribución;
- y una primera capa de calidad de código con **BEM**, **CSS Nesting** y **Stylelint**.

Esta sesión es importante porque aquí dejas atrás el CSS puramente introductorio y empiezas a trabajar como se trabaja de verdad en proyectos: con más precisión, más orden y más criterio técnico.

1. Introducción

En la sesión anterior trabajaste la base del lenguaje: sintaxis, selectores básicos, cascada, especificidad, herencia, unidades, variables, funciones, colores y tipografía.

Esa base era imprescindible. Pero en cuanto una interfaz deja de ser una maqueta mínima, aparecen problemas nuevos:

- seleccionar elementos según su contexto;
- estilizar estados de interacción;
- aplicar estilos a partes concretas del contenido;
- entender cómo ocupan espacio las cajas;
- alinear y repartir bloques;
- y evitar que el CSS se convierta en una acumulación desordenada de reglas.

Todo eso entra en esta sesión.

Idea clave

Si la sesión 7 te enseñó a escribir CSS, esta sesión empieza a enseñarte a **usar CSS con precisión, con estructura y con intención**.

A partir de aquí, CSS deja de ser solo “poner estilos” y empieza a convertirse en una herramienta de construcción de interfaces.

Necesidad real	Herramienta CSS relacionada
Estilizar enlaces dentro de una navegación	Selectores y combinadores
Dar estilo a un botón al pasar el ratón o al recibir foco	Pseudo-clases
Añadir un pequeño detalle visual sin tocar el HTML	Pseudo-elementos
Entender por qué una caja ocupa más de lo esperado	Box model
Repartir botones, tarjetas o bloques en fila	Flexbox
Mantener clases más claras y consistentes	BEM
Evitar errores repetidos de sintaxis o convenciones	Stylelint

En esta sesión no se busca que memorices todos los selectores o todas las posibilidades de Flexbox como una lista infinita. Lo importante es comprender:

- cómo seleccionar con más precisión;
- cómo reaccionar a estados;
- cómo se calcula el espacio de una caja;
- cómo alinear y distribuir elementos;
- y cómo empezar a escribir un CSS más limpio y mantenible.

Referencias del apartado 1

- MDN Web Docs. *CSS selectors*.
 - MDN Web Docs. *The box model*.
 - MDN Web Docs. *Basic concepts of flexbox*.
 - Stylelint Documentation.
-

2. Seleccionar con más precisión

Los selectores básicos —etiqueta, clase e id— son imprescindibles, pero en muchos casos no bastan por sí solos. A menudo necesitas seleccionar elementos en relación con otros, o aprovechar información ya presente en el HTML.

Combinadores

Los combinadores permiten seleccionar elementos según su relación estructural.

Combinador	Ejemplo	Qué selecciona
Descendiente	<code>nav a</code>	Enlaces dentro de <code>nav</code> , aunque no sean hijos directos
Hijo directo	<code>nav > a</code>	Enlaces que son hijos directos de <code>nav</code>
Hermano adyacente	<code>h2 + p</code>	El primer párrafo justo después de un <code>h2</code>
Hermano general	<code>h2 ~ p</code>	Los párrafos hermanos posteriores a un <code>h2</code>

Ejemplos:

```
nav a {
  text-decoration: none;
}

nav > a {
  color: #1d4ed8;
}

h2 + p {
  margin-top: 0;
}

h2 ~ p {
  color: #374151;
}
```

Estos combinadores tienen mucho valor cuando expresan una relación real del HTML. No conviene usarlos solo porque “parecen más potentes”.

 **Buen criterio**

Un selector mejor no es el más largo ni el más rebuscado. Es el que expresa con claridad la relación necesaria sin volver frágil el CSS.

Selectores de atributo

CSS también puede seleccionar según atributos.

Selector	Ejemplo	Qué selecciona
Presencia de atributo	<code>input[required]</code>	Inputs que tienen <code>required</code>
Coincidencia exacta	<code>input[type="email"]</code>	Inputs cuyo <code>type</code> es exactamente <code>email</code>

Ejemplos:

```
input[required] {  
  border-color: #dc2626;  
}  
  
input[type="email"] {  
  background-color: #fff6ff;  
}
```

Estos selectores resultan especialmente útiles en formularios, porque el HTML ya contiene mucha información que puedes aprovechar sin añadir clases innecesarias.

Selectores modernos de apoyo

No necesitas profundizar todavía en todos, pero conviene conocer la idea de algunos selectores modernos porque pueden ayudarte a escribir CSS más claro y evitar repeticiones innecesarias.

`:is()`

`:is()` permite agrupar varias alternativas dentro de un mismo selector sin tener que repetir la regla completa.

Un caso bastante real sería este: quieres aplicar el mismo estilo a distintos elementos interactivos dentro de una cabecera.

```
.header :is(a, button) {  
  font-weight: 600;  
  color: #1f2937;  
}
```

En lugar de escribir algo como:

```
.header a {  
  font-weight: 600;  
  color: #1f2937;  
}  
  
.header button {  
  font-weight: 600;  
  color: #1f2937;  
}
```

`:is()` lo concentra en una sola regla y hace el CSS más compacto.

🔗 Especificidad de `:is()`

`:is()` **no tiene una especificidad fija propia.**

Su especificidad viene dada por el selector más específico que haya dentro de sus paréntesis.

Por ejemplo:

```
.card :is(p, .card__text) {  
  color: #374151;  
}
```

Aquí la especificidad será mayor que si solo estuvieras agrupando etiquetas, porque dentro de `:is()` aparece una clase (`.card__text`).

`:where()`

`:where()` se parece a `:is()` porque también permite agrupar selectores, pero tiene una diferencia muy importante: **su especificidad es siempre 0.**

Eso lo hace especialmente útil para escribir reglas base o reglas amplias que no quieres que “pesen demasiado” en la cascada.

Ejemplo con bastante sentido:

```
:where(main, section, article) p {  
  margin-bottom: 1rem;  
}
```

Esta regla dice: los párrafos que estén dentro de `main`, `section` o `article` tendrán margen inferior.

Sirve bien como estilo base, pero deja espacio para que después puedas sobrescribirlo fácilmente en contextos más concretos.

🔗 Especificidad de `:where()`

Todo lo que metas dentro de `:where()` **no aumenta la especificidad**.
Esa es precisamente su gran ventaja.

Por ejemplo:

```
:where(.card, .panel) h2 {  
  margin-top: 0;  
}
```

Aunque dentro aparezcan clases, `:where()` sigue aportando **especificidad 0** en esa parte.

Eso ayuda mucho a no sobrecargar la cascada con reglas base demasiado fuertes.

`:not()`

`:not()` permite excluir casos concretos. Es útil cuando una regla general se aplica a casi todo, menos a una variante o excepción.

Un ejemplo más realista sería este: tienes varios botones primarios y secundarios, y quieres aplicar un estilo común a todos menos al secundario.

```
.button:not(.button--secondary) {  
  background-color: #2563eb;  
  color: white;  
}
```

Aquí la regla afecta a cualquier elemento con clase `.button` que **no** tenga también `.button--secondary`.

Otro ejemplo razonable:

```
.menu a:not(.menu__link--disabled) {  
  text-decoration: none;  
}
```

Así puedes excluir enlaces desactivados sin duplicar demasiadas reglas.

🔗 Especificidad de `:not()`

`:not()` **no añade una especificidad especial por sí mismo**, pero **sí cuenta la especificidad del selector que contiene dentro**.

Por ejemplo:

```
.button:not(.button--secondary) {  
  color: white;  
}
```

Aquí la especificidad total aumenta porque dentro de `:not()` hay una clase (`.button--secondary`).

En otras palabras: `:not()` no “anula” la especificidad; simplemente excluye, pero el selector interno sigue contando.

🔗 Buen criterio

`:is()`, `:where()` y `:not()` pueden hacer el CSS más expresivo y más limpio, pero conviene usarlos para simplificar, no para construir selectores cada vez más difíciles de leer.

Qué debes retener de esta parte

- `:is()` agrupa selectores y toma la especificidad del selector más fuerte que contenga.
- `:where()` agrupa selectores, pero siempre aporta especificidad 0.
- `:not()` excluye casos concretos, y el selector que contiene dentro sí influye en la especificidad.
- Estas herramientas son útiles cuando reducen repetición y ayudan a controlar mejor la cascada.

Referencias del apartado 2

- MDN Web Docs. *Selectors and combinators*.
 - MDN Web Docs. *Attribute selectors*.
 - MDN Web Docs. *:is()*.
 - MDN Web Docs. *:where()*.
 - MDN Web Docs. *:not()*.
-

3. Estados y partes de los elementos

CSS no solo selecciona elementos por su tipo o su posición. También puede reaccionar a estados y dirigirse a partes concretas de un elemento. Aquí entran las pseudo-clases y los pseudo-elementos.

Pseudo-clases

Las pseudo-clases permiten seleccionar elementos según su estado o posición.

Pseudo-clase	Uso habitual
<code>:hover</code>	Estado al pasar el puntero
<code>:focus</code>	Estado cuando el elemento recibe foco
<code>:focus-visible</code>	Estado de foco visible, especialmente por teclado
<code>:active</code>	Estado mientras el elemento está siendo activado
<code>:visited</code>	Enlaces ya visitados
<code>:checked</code>	Checkboxes o radios seleccionados
<code>:disabled</code>	Controles desactivados
<code>:required</code>	Campos obligatorios
<code>:first-child</code>	Primer hijo
<code>:last-child</code>	Último hijo
<code>:nth-child()</code>	Selección por posición

Ejemplos:


```
button:hover {
  background-color: #1d4ed8;
}

input:focus {
  outline: 2px solid #2563eb;
}

button:focus-visible {
  outline: 3px solid #0ea5e9;
}

input:checked {
  accent-color: #2563eb;
}

tr:nth-child(odd) {
  background-color: #f9fafb;
}
```

Buena práctica

Si vas a personalizar el foco, no elimines la visibilidad del foco sin ofrecer una alternativa clara y accesible.

Pseudo-elementos

Los pseudo-elementos permiten dirigirte a partes concretas del contenido o generar contenido visual adicional.

Pseudo-elemento	Uso habitual
::before	Insertar contenido antes
::after	Insertar contenido después
::placeholder	Estilizar el placeholder
::selection	Estilizar el texto seleccionado

Ejemplos:

```
.badge::before {
  content: "★ ";
}

.external-link::after {
  content: " ↗";
}

input::placeholder {
  color: #9ca3af;
}

::selection {
  background-color: #bfdbfe;
}
```

⚠ Criterio importante

`::before` y `::after` son útiles para detalles visuales o decorativos. No deben sustituir contenido importante que debería existir realmente en el HTML.

Qué debes retener de esta parte

- Las pseudo-clases sirven para estados o posiciones.
- Los pseudo-elementos sirven para partes concretas o contenido añadido.
- Ambas herramientas son muy útiles en interfaces reales, pero conviene usarlas con intención y no por adorno.

Referencias del apartado 3

- MDN Web Docs. *Pseudo-classes*.
- MDN Web Docs. *Pseudo-elements*.

4. Box model, dimensiones y comportamiento de caja

Una de las claves de CSS consiste en entender que cada elemento se comporta como una caja. Esa caja no está formada solo por el contenido visible, sino por varias capas.

El box model

Cada caja puede entenderse en cuatro zonas:

Parte	Qué representa
Contenido	El contenido real del elemento
padding	Espacio interior entre contenido y borde
border	Borde de la caja
margin	Espacio exterior entre la caja y otras cajas

Ejemplo:

```
.card {
  width: 300px;
  padding: 16px;
  border: 1px solid #d1d5db;
  margin: 24px;
}
```

La diferencia entre `padding` y `margin` es fundamental:

Propiedad	Tipo de espacio
padding	Espacio interior
margin	Espacio exterior

Error frecuente

Muchas personas empiezan CSS sin distinguir bien `padding` y `margin`. Eso genera mucha confusión visual y de cálculo.

Dimensiones

También conviene entender que no todo se resuelve con `width` y `height` fijos. En muchos casos tienen más sentido:

- `min-width`
- `max-width`
- `min-height`
- `max-height`

Estas propiedades ayudan a construir interfaces más estables y menos rígidas.

`box-sizing`

La propiedad `box-sizing` controla cómo se calcula el tamaño de la caja.

Valor	Comportamiento
<code>content-box</code>	<code>width</code> y <code>height</code> afectan solo al contenido
<code>border-box</code>	<code>width</code> y <code>height</code> incluyen contenido, <code>padding</code> y <code>border</code>

Ejemplo:

```
* {  
  box-sizing: border-box;  
}
```

Buena práctica habitual

Usar `box-sizing: border-box` como base global suele facilitar mucho el trabajo de maquetación.

`overflow`

Cuando el contenido supera el espacio disponible, entra en juego `overflow`.

Valores frecuentes:

- `visible`
- `hidden`
- `auto`
- `scroll`

Ejemplo:

```
.panel {  
  max-height: 300px;  
  overflow: auto;  
}
```

Esto es útil en paneles, listados largos, cajas de código o bloques con tamaño limitado.

display

La propiedad `display` influye en cómo se comporta un elemento dentro del flujo del documento.

Valor	Comportamiento general
<code>block</code>	Ocupa el ancho disponible y provoca salto de línea
<code>inline</code>	No provoca salto y ocupa solo lo necesario
<code>inline-block</code>	Se comporta en línea, pero admite mejor dimensiones
<code>none</code>	Oculto el elemento y lo saca del flujo visual

Ejemplos:

```
div {  
  display: block;  
}  
  
span {  
  display: inline;  
}  
  
.button-chip {  
  display: inline-block;  
}  
  
.hidden {  
  display: none;  
}
```

Bordes y esquinas

Los bordes y radios son muy comunes en componentes reales.

```
.card {  
  border: 1px solid #d1d5db;  
  border-radius: 12px;  
}
```

Esto aparece continuamente en:

- tarjetas;
- botones;
- inputs;
- paneles;
- etiquetas;
- modales.

Qué debes retener de esta parte

- Todo elemento se comporta como una caja.
- `padding` y `margin` no son lo mismo.
- El tamaño real de una caja depende de su modelo de cálculo.
- `box-sizing: border-box` suele ser una buena base.
- `overflow` controla qué pasa cuando el contenido desborda.
- `display` cambia profundamente el comportamiento de un elemento.

Referencias del apartado 4

- MDN Web Docs. *The box model*.
 - MDN Web Docs. *box-sizing*.
 - MDN Web Docs. *overflow*.
 - MDN Web Docs. *display*.
 - MDN Web Docs. *border*.
 - MDN Web Docs. *border-radius*.
-

5. Flexbox

Flexbox es uno de los sistemas más útiles de CSS para distribuir y alinear elementos. Sirve especialmente bien cuando quieres organizar contenido en una sola dimensión: en fila o en columna.

Antes de Flexbox, alinear elementos horizontalmente o repartir espacio entre bloques podía ser incómodo y requerir soluciones poco intuitivas. Flexbox resuelve de forma más clara problemas como:

- centrar elementos;
- repartir espacio;
- crear barras de navegación;
- alinear botones;

- distribuir tarjetas;
- construir formularios sencillos en fila o columna.

Activar Flexbox

Para convertir un contenedor en flexible:

```
.container {  
  display: flex;  
}
```

A partir de ese momento, sus hijos pasan a organizarse como elementos flexibles.

Eje principal y eje cruzado

Flexbox funciona con dos ejes:

- el **eje principal**;
- el **eje cruzado**.

Por defecto:

- el eje principal va en horizontal;
- el eje cruzado va en vertical.

La orientación puede cambiar con `flex-direction`.

Propiedades principales del contenedor flex

Propiedad	Función
<code>flex-direction</code>	Define la dirección del eje principal
<code>justify-content</code>	Alinea elementos a lo largo del eje principal
<code>align-items</code>	Alinea elementos a lo largo del eje cruzado
<code>gap</code>	Define el espacio entre elementos
<code>flex-wrap</code>	Permite salto de línea si no caben

Ejemplo:

```
.container {  
  display: flex;
```

```
flex-direction: row;
justify-content: space-between;
align-items: center;
gap: 16px;
flex-wrap: wrap;
}
```

Valores frecuentes de `justify-content` :

- `flex-start`
- `center`
- `flex-end`
- `space-between`
- `space-around`
- `space-evenly`

Valores frecuentes de `align-items` :

- `stretch`
- `flex-start`
- `center`
- `flex-end`

Buena práctica

Cuando necesitas espacio entre elementos flex, `gap` suele ser más limpio que repartir márgenes manualmente.

Propiedades útiles de los hijos flex

Propiedad	Función
<code>flex-grow</code>	Indica cuánto puede crecer un elemento
<code>flex-shrink</code>	Indica cuánto puede reducirse
<code>flex-basis</code>	Define un tamaño base inicial
<code>flex</code>	Forma abreviada muy usada

Ejemplos:


```
.card {  
  flex-grow: 1;  
}  
  
.card {  
  flex-basis: 240px;  
}  
  
.card {  
  flex: 1;  
}
```

Casos de uso reales con Flexbox

Flexbox resulta muy útil en:

- barras de navegación;
- grupos de botones;
- cabeceras con logo y enlaces;
- formularios simples;
- listados de tarjetas;
- filas de información;
- distribución de bloques internos.

Ejemplos:

```
.navbar {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
}  
  
.actions {  
  display: flex;  
  gap: 12px;  
}  
  
.card-list {  
  display: flex;  
  gap: 20px;  
  flex-wrap: wrap;  
}
```

Qué debes retener de esta parte

- Flexbox organiza elementos en una dimensión.
- `justify-content` trabaja sobre el eje principal.
- `align-items` trabaja sobre el eje cruzado.
- `gap` y `flex-wrap` son especialmente útiles en maquetación real.
- Flexbox resuelve muchos problemas de alineación con mucha más claridad que soluciones antiguas.

Referencias del apartado 5

- MDN Web Docs. *Basic concepts of flexbox*.
 - MDN Web Docs. *Aligning items in a flex container*.
 - MDN Web Docs. *flex*.
-

6. BEM, nesting y Stylelint

Esta parte introduce una idea importante: escribir CSS no consiste solo en que “funcione”, sino también en que **siga siendo legible y mantenible** cuando el proyecto crece.

BEM

BEM es una metodología de nombrado de clases. Sus siglas vienen de:

- Block
- Element
- Modifier

Ejemplo:

```
.card {}  
.card__title {}  
.card__text {}  
.card--featured {}
```

Aquí:

- `.card` es el bloque;
- `.card__title` y `.card__text` son elementos del bloque;
- `.card--featured` es una variante del bloque.

BEM ayuda a:

- hacer más claras las clases;
- reducir ambigüedad;
- entender mejor la relación entre partes;
- y evitar nombres genéricos o poco mantenibles.

Criterio importante

BEM no es una religión ni una obligación absoluta. Su valor está en aportar claridad y consistencia, no en complicar nombres sin necesidad.

CSS Nesting

CSS Nesting permite anidar reglas de forma nativa, sin recurrir a preprocesadores como Sass.

Ejemplo:

```
.card {  
  padding: 16px;  
  border-radius: 12px;  
  
  & .card__title {  
    font-weight: 700;  
  }  
  
  & .card__link:hover {  
    text-decoration: underline;  
  }  
}
```

Esto puede mejorar la lectura si el anidado es corto y claro.

Pero también puede empeorar el CSS si se usa sin control.

Buen criterio

Anidar no significa “meter todo dentro de todo”. Si el CSS se vuelve profundo o difícil de leer, el nesting deja de ayudar.

Stylelint

Stylelint es un linter para CSS. Sirve para revisar automáticamente tu código y detectar:

- errores de sintaxis;
- convenciones inconsistentes;
- propiedades duplicadas;
- nombres problemáticos;
- y malas prácticas configurables.

No necesitas en esta sesión dominarlo por completo, pero sí entender su papel: igual que en JavaScript existen herramientas para vigilar la calidad del código, en CSS también conviene apoyarse en ellas.

Una configuración mínima de proyecto puede ayudarte a:

- mantener formato consistente;
- detectar errores antes;
- y reforzar convenciones de equipo.

Idea práctica

Stylelint no “escribe por ti”, pero te ayuda a que el CSS sea más consistente y menos propenso a errores repetidos.

Qué debes retener de esta parte

- El CSS también necesita convenciones y estructura.
- BEM ayuda a nombrar clases con más claridad.
- CSS Nesting puede mejorar la lectura si se usa con medida.
- Stylelint es una herramienta útil para reforzar calidad y consistencia.

Referencias del apartado 6

- MDN Web Docs. *CSS nesting*.
- BEM Methodology.
- Stylelint Documentation.

7. Análisis técnico avanzado y cierre

En esta sesión has trabajado un CSS más preciso y más estructurado. Eso implica un cambio importante: ya no basta con “poner estilos”, ahora necesitas entender mejor cómo se

relacionan selector, estructura, caja, layout y mantenimiento.

Muchos problemas en CSS no vienen de propiedades raras, sino de fundamentos mal gestionados:

- selectores demasiado complejos o frágiles;
- pseudo-clases usadas sin pensar en accesibilidad;
- pseudo-elementos usados para suplir contenido que debería estar en HTML;
- mala comprensión del box model;
- abuso de `margin` para resolver cualquier problema;
- o Flexbox usado sin entender bien sus ejes.

A eso se suma otro problema muy habitual: el CSS crece rápido. Si no empiezas pronto a pensar en claridad, nombres y herramientas de calidad, es fácil que el archivo de estilos se vuelva difícil de mantener.

Esta sesión prepara además lo que vendrá después:

- Flexbox te permite resolver muchos componentes y distribuciones simples;
- Grid ampliará esa lógica a layouts más completos;
- responsive design se apoyará en estas bases;
- y BEM, nesting y Stylelint te preparan para escribir CSS más cercano a un entorno de proyecto real.

Qué debes retener de esta sesión

- Los selectores avanzados permiten trabajar con más precisión.
- Las pseudo-clases sirven para estados y las pseudo-elementos para partes concretas o contenido añadido.
- Todo elemento se comporta como una caja con contenido, `padding`, `border` y `margin`.
- `overflow`, `display` y `box-sizing` afectan mucho al comportamiento visual.
- Flexbox es una herramienta esencial para distribuir y alinear elementos en una dimensión.
- Bordes y radios son parte habitual del trabajo con componentes.
- BEM ayuda a nombrar clases con más claridad.
- CSS Nesting puede mejorar lectura si no se abusa de él.
- Stylelint ayuda a mantener consistencia y detectar errores.
- Es mejor escribir menos CSS, pero más claro, que mucho CSS difícil de sostener.

Referencias documentales generales

- 1. MDN Web Docs. *CSS selectors*.
- 2. MDN Web Docs. *Selectors and combinators*.
- 3. MDN Web Docs. *Pseudo-classes*.
- 4. MDN Web Docs. *Pseudo-elements*.
- 5. MDN Web Docs. *The box model*.
- 6. MDN Web Docs. *overflow*.
- 7. MDN Web Docs. *display*.
- 8. MDN Web Docs. *Basic concepts of flexbox*.
- 9. MDN Web Docs. *CSS nesting*.
- 10. BEM Methodology.
- 11. Stylelint Documentation.

Mini glosario

Término	Definición breve
Combinador	Relación entre selectores en CSS
Pseudo-clase	Selector basado en estado o posición
Pseudo-elemento	Selector aplicado a una parte concreta del elemento
Box model	Modelo de caja de los elementos en CSS
overflow	Control del contenido cuando desborda
display	Propiedad que define cómo se comporta una caja
Flexbox	Sistema de distribución y alineación en una dimensión
BEM	Metodología de nombrado de clases
Nesting	Anidado de reglas CSS
Linter	Herramienta que analiza código y detecta errores o incoherencias

Fragmento de ejemplo integrador

HTML

```
<section class="course-list">
  <article class="course-card course-card--featured">
```

```

    <h2 class="course-card__title">HTML y semántica</h2>
    <p class="course-card__text">Aprende a estructurar correctamente una
página web.</p>
    <a class="course-card__link" href="#">Ver detalles</a>
  </article>

  <article class="course-card">
    <h2 class="course-card__title">CSS y maquetación</h2>
    <p class="course-card__text">Empieza a construir interfaces limpias y
mantenibles.</p>
    <a class="course-card__link" href="#">Ver detalles</a>
  </article>
</section>

```

CSS

```

.course-list {
  display: flex;
  gap: 20px;
  flex-wrap: wrap;
}

.course-card {
  flex: 1 1 260px;
  padding: 16px;
  border: 1px solid #d1d5db;
  border-radius: 12px;
  overflow: hidden;

  & .course-card__title {
    font-weight: 700;
  }

  & .course-card__link:hover {
    text-decoration: underline;
  }
}

.course-card--featured {
  border-color: #2563eb;
}

~~~

```